

✖ ToxicGuard V3 — Transformer (Deep Learning) Notebook

Bu versiyonda klasik Makine Öğrenmesi (TF-IDF + Logistic Regression) altyapısını geride bırakıp, **Bağlamı (Context), ironiyi ve üstü kapalı tehditleri ('boğaz keseceğim' vb.) anlayabilen devasa dil modelleri (LLM - Transformer)** olan **DistilBERT** / **RoBERTa** mimarisini eğiteceğiz.

💡 V3 Yenilikleri & Özellikleri

1. **Cache / Çakışma Sorunu Yok:** Bu versiyonda eski `X_train.pkl` (TF-IDF) cache dosyaları hiç kullanılmaz. Model kelimeleri değil, baştan sona cümle bağlamını direkt işler. Hiçbir cache temizliğine gerek yoktur.
2. **Üzerine Yazma Engellendi:** Kaydedilen grafikler ve model ağırlıkları `v3_` önekiyle kaydedilir. Eski dosyalarınız (V1, V2) güvendedir.
3. **Metin Temizliği İPTAL:** Transformerlar noktalama işaretlerinden, cümlelerin büyük/küçük harf oranından BİLE duygu çıkarımı yapar. Dolayısıyla agresif temizlik (`clean_text`) yapmayıp metni ham haliyle modele vereceğiz.

✖ 🔧 BÖLÜM 1 — Kurulum & Drive Bağlantısı

```
# HÜCRE 1: Gerekli Paketlerin Kurulumu
!pip install transformers datasets evaluate accelerate --quiet
print("✅ Hugging Face paketleri kuruldu!")
```

84.1/84.1 kB 3.1 MB/s eta 0:00:00
✅ Hugging Face paketleri kuruldu!

```
# HÜCRE 2: Google Drive Bağlantısı
import os
from google.colab import drive
drive.mount('/content/drive')

BASE = '/content/drive/MyDrive/ToxicGuard'
MODELS_DIR = os.path.join(BASE, 'models')
DATA_DIR = os.path.join(BASE, 'data')
RESULTS_DIR = os.path.join(BASE, 'reports', 'model_results')
EDA_DIR = os.path.join(BASE, 'reports', 'eda_plots')

for d in [MODELS_DIR, DATA_DIR, RESULTS_DIR, EDA_DIR]:
    os.makedirs(d, exist_ok=True)

print("✅ Klasör yolları ayarlandı!")
```

Mounted at /content/drive
✅ Klasör yolları ayarlandı!

```
# HÜCRE 3: Kütüphaneleri Dahil Etme
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import torch
from datasets import Dataset
from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
    TrainingArguments,
    Trainer,
    EvalPrediction
)
from sklearn.metrics import f1_score, roc_auc_score, accuracy_score

LABEL_COLS = ['toxic', 'severe_toxic', 'obscene', 'threat', 'insult', 'identity_hate']
LABEL_TR = {'toxic': 'Toksik', 'severe_toxic': 'Ağır Toksik', 'obscene': 'Müstehcen',
            'threat': 'Tehdit', 'insult': 'Hakaret', 'identity_hate': 'Kimlik Nefreti'}

# GPU Aktif mi?
print(f"Cihaz: {'GPU Aktif! 🚀' if torch.cuda.is_available() else 'CPU (Yavaş) ⚠️ Runtime -> Change Runtime Type menüsünde"}")
```

Cihaz: GPU Aktif! 🚀

✖ 📁 BÖLÜM 2 — Veri Yükleme ve V3 Grafik Çizimleri

```
# HÜCRE 4: Veriyi Yükle (Temizleme YAPMADAN Orijinal Cümleler Kullanılacak)
TRAIN_CSV = os.path.join(DATA_DIR, 'train.csv')
df = pd.read_csv(TRAIN_CSV)

print(f"Veri Seti Yüklendi: {df.shape}")

# V3 EDA Plot: Etiket Dağılımını v3_ önekiyle Kaydet
fig, ax = plt.subplots(figsize=(10, 5))
counts = df[LABEL_COLS].sum().sort_values(ascending=False)
sns.barplot(x=[LABEL_TR[c] for c in counts.index], y=counts.values, palette='viridis', ax=ax)
ax.set_title('V3: Tam Kapsamlı (Ham Veri) Toksik Dağılımı')

plt.savefig(os.path.join(EDA_DIR, 'v3_01_label_distribution.png'), bbox_inches='tight')
plt.show()

print("✅ EDA plot kaydedildi: v3_01_label_distribution.png")
```

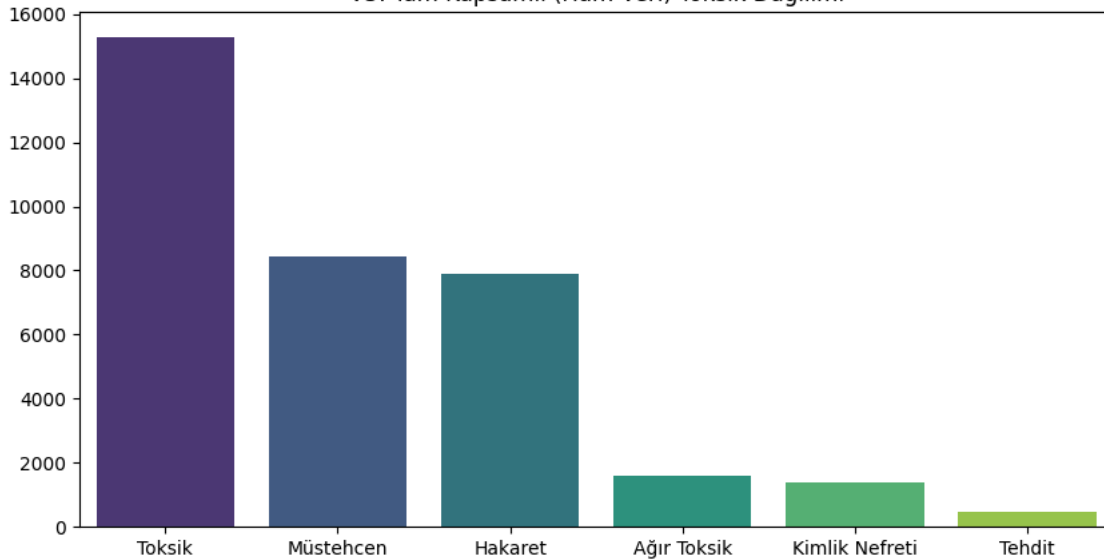
Veri Seti Yüklendi: (159571, 8)

/tmp/ipykernel_1084/1459409279.py:10: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and

```
sns.barplot(x=[LABEL_TR[c] for c in counts.index], y=counts.values, palette='viridis', ax=ax)
```

V3: Tam Kapsamlı (Ham Veri) Toksik Dağılımı



✅ EDA plot kaydedildi: v3_01_label_distribution.png

```
# HÜCRE 5: Hugging Face Dataset Formatına Çevirme
# Transformer eğitimleri için veriyi Dictionary/PyTorch Tensor formatına çevireceğiz

# NOT: Eğer eğitim çok uzun sürerse buradaki 'sample_size' ile veriyi kırabilirsiniz.
# Şimdilik tam veri setinin küçük bir prototipiyle hızlıca başlıyoruz, dilerseniz artırabilirsiniz.
USE_FULL_DATA = False
```

```
if not USE_FULL_DATA:
    print("Hızlı sonuç için veri setinden dengeli bir örneklem alınıyor...")
    toxic = df[df[LABEL_COLS].sum(axis=1) > 0]
    safe = df[df[LABEL_COLS].sum(axis=1) == 0].sample(len(toxic) * 2, random_state=42)
    df_train = pd.concat([toxic, safe]).sample(frac=1, random_state=42).reset_index(drop=True)
else:
    df_train = df.copy()
```

```
print(f"Eğitime girecek nihai veri boyutu: {df_train.shape}")
```

```
# HuggingFace num_labels mantığı
labels = df_train[LABEL_COLS].values.astype(float)
texts = df_train['comment_text'].tolist()

dataset = Dataset.from_dict({'text': texts, 'labels': labels})
dataset = dataset.train_test_split(test_size=0.1, seed=42)
print(dataset)
```

Hızlı sonuç için veri setinden dengeli bir örneklem alınıyor...

Eğitime girecek nihai veri boyutu: (48675, 8)

```
DatasetDict({
  train: Dataset({
    features: ['text', 'labels'],
    num_rows: 43807
  })
  test: Dataset({
    features: ['text', 'labels'],
```

```
num_rows: 4868
    })
})
```

📁 BÖLÜM 3 — Model ve Tokenizer Hazırlığı (DistilBERT)

```
# HÜCRE 6: DistilBERT Tokenizer (Hafif ve Hızlı)
MODEL_NAME = "distilbert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
```

```
def tokenize_fn(examples):
    return tokenizer(examples['text'], padding="max_length", truncation=True, max_length=128)

print("Metinler makine diline çevriliyor (Tokenization, ~1dk sürer)...")
tokenized_dataset = dataset.map(tokenize_fn, batched=True)
tokenized_dataset.set_format('torch', columns=['input_ids', 'attention_mask', 'labels'])
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:93: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and fast
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to
config.json: 100% 483/483 [00:00<00:00, 7.34kB/s]

tokenizer_config.json: 100% 48.0/48.0 [00:00<00:00, 1.74kB/s]

vocab.txt: 232k/? [00:00<00:00, 881kB/s]

tokenizer.json: 466k/? [00:00<00:00, 1.08MB/s]
Metinler makine diline çevriliyor (Tokenization, ~1dk sürer)...
Map: 100% 43807/43807 [00:33<00:00, 1438.16 examples/s]
Map: 100% 4868/4868 [00:04<00:00, 1065.11 examples/s]
```

```
# HÜCRE 7: Sınıflandırma Modelini Yükle (Multi-Label)
model = AutoModelForSequenceClassification.from_pretrained(
    MODEL_NAME,
    num_labels=len(LABEL_COLS),
    problem_type="multi_label_classification"
)
print("✅ Model başarıyla yüklendi.")
```

```
model.safetensors: 100% 268M/268M [00:03<00:00, 241MB/s]

Loading weights: 100% 100/100 [00:00<00:00, 260.14it/s, Materializing param=distilbert.transformer.layer.5.sa_layer_norm.weight]
DistilBertForSequenceClassification LOAD REPORT from: distilbert-base-uncased
Key | Status |
-----+-----+
vocab_layer_norm.bias | UNEXPECTED |
vocab_projector.bias | UNEXPECTED |
vocab_transform.weight | UNEXPECTED |
vocab_transform.bias | UNEXPECTED |
vocab_layer_norm.weight | UNEXPECTED |
classifier.bias | MISSING |
pre_classifier.bias | MISSING |
classifier.weight | MISSING |
pre_classifier.weight | MISSING |

Notes:
- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING :those params were newly initialized because missing from the checkpoint. Consider training on your downstream task
✅ Model başarıyla yüklendi.
```

📁 BÖLÜM 4 — Trainer API ile Multi-Label Eğitim

```
# HÜCRE 8: Metrik Fonksiyonu
def compute_metrics(p: EvalPrediction):
    preds = p.predictions[0] if isinstance(p.predictions, tuple) else p.predictions
    # Sigmoid ile olasılıklara çevir, 0.5 threshold'tan kes.
    # (Gelişmiş V3 Dinamik eşik hesaplanabilir ancak Transformerlar net karar verdiği için 0.5 ile başlanır.)
    probs = torch.sigmoid(torch.tensor(preds)).numpy()
    y_pred = (probs > 0.5).astype(int)
    y_true = p.label_ids
```

```

f1_macro = f1_score(y_true, y_pred, average='macro', zero_division=0)
roc_auc = roc_auc_score(y_true, probs, average='macro', multi_class='ovr')

return {'f1_macro': f1_macro, 'roc_auc': roc_auc}

# HÜCRE 9: Eğitim Konfigürasyonları
training_args = TrainingArguments(
    output_dir=os.path.join(MODELS_DIR, 'distilbert_v3_checkpoints'),
    eval_strategy="epoch",
    save_strategy="epoch",
    learning_rate=2e-5,
    per_device_train_batch_size=32,
    per_device_eval_batch_size=32,
    num_train_epochs=2, # Çok epok ezberleme (overfitting) yapar, 2 veya 3 ideal.
    weight_decay=0.01,
    load_best_model_at_end=True,
    metric_for_best_model="f1_macro",
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_dataset['train'],
    eval_dataset=tokenized_dataset['test'],
    compute_metrics=compute_metrics
)

```

```

# HÜCRE 10: Eğitimi Başlat!
print("🚀 Eğitim Başlıyor! (Bu işlem cihazınızın hızına göre 5-20 dk sürebilir)")
trainer.train()
print("✅ Eğitim Tamamlandı!")

```

🚀 Eğitim Başlıyor! (Bu işlem cihazınızın hızına göre 5-20 dk sürebilir)

[2738/2738 16:29, Epoch 2/2]

Epoch	Training Loss	Validation Loss	F1 Macro	Roc Auc
1	0.117545	0.102543	0.594440	0.975516
2	0.096265	0.100802	0.693254	0.978148

Writing model shards: 100% 1/1 [00:00<00:00, 1.25it/s]

Writing model shards: 100% 1/1 [00:03<00:00, 3.13s/it]

There were missing keys in the checkpoint model loaded: ['distilbert.embeddings.LayerNorm.weight', 'distilbert.embeddings.LayerNorm.bias']

There were unexpected keys in the checkpoint model loaded: ['distilbert.embeddings.LayerNorm.beta', 'distilbert.embeddings.LayerNorm.running_mean', 'distilbert.embeddings.LayerNorm.running_var']

✅ Eğitim Tamamlandı!

📁 BÖLÜM 5 — Modeli V3 Olarak Kaydet

```

# HÜCRE 11: Modeli Kaydetme
v3_model_path = os.path.join(MODELS_DIR, 'toxicguard_v3_transformer')

# Eski Transformer kayıtlarıyla çakışmamak için direkt dizine kaydediyoruz
trainer.save_model(v3_model_path)
tokenizer.save_pretrained(v3_model_path)

print(f"🎉 HARİKA! State-of-the-Art Transformer Modeliniz şuraya kaydedildi:\n{v3_model_path}")
print("\nUygulamanız (app.py) içerisinde bu dizini göstererek devasa dil modelinizi kullanabilirsiniz!")

```

Writing model shards: 100% 1/1 [00:01<00:00, 1.00it/s]

🎉 HARİKA! State-of-the-Art Transformer Modeliniz şuraya kaydedildi:
/content/drive/MyDrive/ToxicGuard/models/toxicguard_v3_transformer

Uygulamanız (app.py) içerisinde bu dizini göstererek devasa dil modelinizi kullanabilirsiniz!

